A Network in Code

This module builds networks in code with the `igraph` library, and asks them questions.

In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: build a network, then edit a copy

/// details | 📋 Task type: lh-task

Task:

- Build the network in igraph. It has 7 nodes, numbered 0 to 6, and the 9 edges in `edge_list`.
- Ask the network how many nodes and how many edges it has.
- Make a copy of the network.
- Add one node to the copy, and one edge from the new node to node 6.
- Delete node 0 from the copy.
- Find the new number of the node that used to be node 2.

The igraph calls are in the walkthrough, steps 1 to 3. ///

```
7 9 1
```



Building a network, and editing a copy

```
g = ig.Graph(n=7, edges=edge_list)
g.vcount(), g.ecount()        # 7, 9
edited = g.copy()
edited.add_vertices(1)        # the new node is number 7
edited.add_edges([(7, 6)])    # one edge, from 7 to 6
edited.delete_vertices([0])   # node 0, and both of its edges
```

A list of edges only mentions nodes that have at least one edge. `n` sets how many nodes there are, so a node with no edges still exists. `delete_vertices()` removes the node and its edges: the copy goes from 8 nodes and 10 edges to **7 nodes and 8 edges**. Every node after the deleted one moves down by one number, so the node that was number 2 is now number **1**. Save node numbers after deleting, or use node names (`g.vs["name"]`) instead.

In [ ]:
```
_d = ig.Graph(n=4, edges=[(0, 1), (1, 2), (2, 3)])
_d.vs["name"] = ["a", "b", "c", "d"]
_d.delete_vertices([0])
mo.vstack([
    mo.md(r"""### 🧭 Detour: node names survive deleting

Yes — store the label as a **vertex attribute** with `g.vs["name"]`. A name
is a string that travels with its node. When a node is deleted, the
surviving nodes keep their names, even though their **numbers** shift down.

Below, the four nodes are named a, b, c, d, then node 0 (named a) is
deleted. The survivors are still b, c, d."""),
    draw_net(_d, title="names after deleting a"),
])
```

 No description has been provided for this image

In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: the network as a square of numbers

▶ 📋 Task

```
[[0 1 1 0 0 0 0]
 [1 0 1 1 0 0 0]
 [1 1 0 1 0 0 0]
 [0 1 1 0 1 0 0]
 [0 0 0 1 0 1 1]
 [0 0 0 0 1 0 1]
 [0 0 0 0 1 1 0]]
row sums: [2 3 3 3 3 2 2]
degrees:  [2, 3, 3, 3, 3, 2, 2]
mean degree: 2.5714285714285716
```

Adjacency matrix, row sums and degree

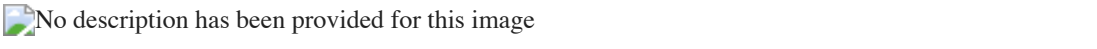
```
A = np.array(g.get_adjacency().data)   # 7 by 7
A.sum(axis=1)                          # [2, 3, 3, 3, 3, 2, 2]
g.degree()                             # [2, 3, 3, 3, 3, 2, 2]
np.mean(g.degree())                    # 2.571
```

The adjacency matrix $\mathbf{A}$ has $A_{ij} = 1$ when nodes i and j share an edge, and $A_{ij} = 0$ otherwise. `axis=1` adds along each row. The network has no direction, so $\mathbf{A}$ is symmetric and `axis=0` gives the same numbers. Each row total is the **degree** of that node, its number of edges. All the entries add up to $2M = 18$: each of the $M = 9$ edges is counted once from each end. The **mean degree** is $\langle k \rangle = 18/7 \approx 2.571$. A network with N nodes has an $N \times N$ matrix; at $N = 10^6$ that is 10^{12} entries, almost all zero. `g.degree()` reads igraph's own list of each node's neighbours and never builds the matrix.

In [ ]:
```
mo.vstack([
    mo.md(r"""### 🧭 Detour: igraph keeps lists, not a full matrix

igraph stores each node's **neighbours** as a list — an **adjacency list**
—
rather than a whole matrix of ones and zeros. So `g.degree()` only reads
the
length of each node's neighbour list. For a large, sparse network this
stays cheap, while a full $N \times N$ matrix would be mostly zeros.

Below: node 2's neighbour list is `[0, 1, 3]`, so its degree is 3."""),
    draw_net(g, highlight=(2,), title="node 2 and its neighbours"),
])
```


No description has been provided for this image

In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: routes, and the pieces left after a cut

▶ 📋 Task

```
8 routes; shortest: [0, 1, 3, 4, 6] crosses 4 edges
sizes: [4, 1, 2] biggest: 4
```



Routes, and the pieces after a cut

```
g.get_all_simple_paths(0, to=6)   # 8 routes
g.get_shortest_paths(0, to=6)[0]  # [0, 1, 3, 4, 6]
cut = g.copy()
cut.delete_edges(cut.incident(4))
[len(c) for c in cut.connected_components()]  # [4, 2, 1]
```

A **simple path** never visits a node twice. There are 8 from node 0 to node 6. `get_shortest_paths` gives a **list** of routes, because two routes, `[0, 1, 3, 4, 6]` and `[0, 2, 3, 4, 6]`, both cross 4 edges. A **connected component** is a group of nodes that can all reach each other along edges. Cutting node 4's edges leaves three: `[4, 2, 1]`. `[4, 2, 1]` and `[6, 1, 1]` are both three pieces, so the count does not show the damage. The size of the **largest component** does.

In [ ]:
```
_cut = g.copy()
_cut.delete_edges(_cut.incident(4))
_del = g.copy()
_del.delete_vertices([4])
mo.vstack([
    mo.md(r"""### 🧭 Detour: articulation point — cut edges vs delete the
node

Node 4 is an **articulation point** (a cut vertex): removing it splits the
connected network into more than one piece.

Cutting its edges leaves node 4 standing as an isolated node — **3 pieces,
7 nodes**. Deleting the node removes it entirely — **2 pieces, 6 nodes**.
The total changes, so the connectivity number (largest piece ÷ total) is
not
the same."""),
    mo.hstack([
        draw_net(_cut, closed=(4,), title="edges cut: node 4 stays"),
        draw_net(_del, title="node 4 deleted"),
    ]),
])
```



In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
```

```
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: connectivity and the R-index

▶ 📋 Task



Connectivity, the robustness curve and R

```
def connectivity(g):
    groups = g.connected_components()
    return max(len(group) for group in groups) / g.vcount()

def r_index(closed, connectivities):
return np.mean(connectivities)
```

Connectivity is the share of a network's nodes in its largest connected group. A whole network has connectivity 1. A closed node stays in the network with no edges, so the total does not change. Running connectivity after every closure gives a **robustness curve**. The **R-index** is its average height:

$$R = \frac{1}{T}\sum_{t=1}^{T} \text{connectivity}_t$$

After closing a share f of the nodes, connectivity is at most $1 - f$, so over the range $f = 0$ to 0.6 the best case is $R = 0.7$, and a network that falls apart at once gives R close to 0. This network gives $R = 0.338$. A reported R always comes with its range.

In [ ]:
```
import numpy as _np
_x = _np.linspace(0, 0.6, 61)
_gradual = 1 - _x                # falls steadily
_sudden = _np.where(_x < 0.45, 1.0, 1 - (_x - 0.45) / 0.15 * 0.9)  # holds,
then collapses
_fig, _ax = plt.subplots(figsize=(6, 3.5))
_ax.plot(_x, _gradual, color="#B14434", label="falls gradually")
_ax.plot(_x, _sudden, color="#3959A6", label="holds, then collapses")
_ax.set_xlabel("share closed")
_ax.set_ylabel("connectivity")
_ax.set_ylim(0, 1.05)
_ax.legend()
_ax.set_title("same R, different shapes")
_fig.tight_layout()
mo.vstack([
    mo.md(r"""### 🧭 Detour: the same R can hide very different curves
```

```
R is the **average height** of the robustness curve, so it hides the shape.
Two networks can share the same R while one falls gradually from the start
and the other stays near 1 for a long time, then collapses suddenly."""),
    _fig,
])
```


No description has been provided for this image

In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: grow a network by preferential attachment

▶ 📋 Task

```
2000 nodes and 5994 edges
```


No description has been provided for this image

Preferential attachment, written by hand

```
rng = np.random.default_rng(42)
m = 3
pa = ig.Graph.Full(m + 1)
for new in range(m + 1, 2000):
    k_now = np.array(pa.degree())
    p = k_now / k_now.sum()
    targets = rng.choice(pa.vcount(), size=m, replace=False, p=p)
    pa.add_vertices(1)
    pa.add_edges([(new, t) for t in targets])
```

Each new node links to node i with probability $p_i = k_i / \sum_j k_j$. This is **preferential attachment**: a node that already has many edges is more likely to get the next one. It is the rule of the **Barabási–Albert model**, and `ig.Graph.Barabasi(2000, 3)` grows the same kind of network. The oldest nodes had the most time to collect edges, so they become the hubs: in a network of 2000 nodes the largest degree is above 100. When each new node picks at random instead, the largest degree is about 26, and there are no hubs.

In [ ]:
```
import numpy as _np
_k = _np.array(pa.degree())
_fig, _ax = plt.subplots(figsize=(6, 3.5))
_ax.scatter(_np.arange(pa.vcount()), _k, s=4, color="#3959A6", alpha=0.5)
_ax.set_xlabel("node number (order it arrived)")
```

```
_ax.set_ylabel("degree")
_ax.set_title("old nodes (small numbers) hold the most edges")
_fig.tight_layout()
mo.vstack([
    mo.md(r"""### 🧭 Detour: growth and preferential attachment work
together

In the Barabási–Albert model, two rules act at once. **Growth** means nodes
arrive one at a time, so some nodes are older than others. **Preferential
attachment** means a node with more edges is more likely to be picked next.

Each node's number is the order it arrived. Older nodes — the small numbers
—
had more chances to be picked, and their early lead grew because the rich
get
richer. That is why the big degrees sit on the left."""),
    _fig,
])
```

 No description has been provided for this image

In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: the degree distribution of a network with hubs

▶ 📋 Task

```
alpha from the slope:    3.1152509303738243
alpha from powerlaw.Fit: 2.9208179999359514
```

 No description has been provided for this image

The degree distribution of a Barabási–Albert network

```
ba = ig.Graph.Barabasi(2000, 3)
k = np.array(ba.degree())
ks = np.unique(k)
ccdf = np.array([(k >= x).mean() for x in ks])
slope = np.polyfit(np.log(ks), np.log(ccdf), 1)[0]
alpha = 1 - slope      # about 3.1

fit = powerlaw.Fit(k, discrete=True, xmin=10)
alpha_fit = fit.power_law.alpha   # about 3.0
```

In a Barabási–Albert network, each new node links to nodes that already have many edges more often than to nodes with few. The degree distribution then follows a **power law**, $p(k) \propto k^{-\alpha}$, with $\alpha = 3$. The share of nodes with degree **at least** k follows $k^{-(\alpha-1)}$, a straight line of slope $1 - \alpha$ on log-log axes. It is much less noisy than the plain distribution: with 2000 nodes, fitting the plain distribution gives an exponent between 2.2 and 2.8, while the cumulative one gives about 3. A straight line fitted to a log-log plot is a first look, not an estimate to report. `powerlaw.Fit` estimates α by **maximum likelihood**, from the degrees at or above a lower cut-off $k_{\min}$. The power law describes the tail, so the cut-off matters: with $k_{\min} = 10$ it gives about 3.0, and if the package is left to pick a low $k_{\min}$ on 2000 nodes it gives about 2.6 to 2.7. The method, and why a straight line on a log-log plot is not enough, is in Clauset, Shalizi and Newman (2009), "Power-law distributions in empirical data", *SIAM Review* 51(4).

In [ ]:
```
import numpy as _np
_k = _np.array(ba.degree())
_ks = _np.unique(_k)
_ccdf = _np.array([(_k >= x).mean() for x in _ks])
_tail = _ks >= 10
_fig, _ax = plt.subplots(figsize=(6, 3.5))
_ax.loglog(_ks[~_tail], _ccdf[~_tail], "o", color="#DAB167", label="low
degrees (bend)")
_ax.loglog(_ks[_tail], _ccdf[_tail], "o", color="#3959A6", label="tail k ≥
10 (straight)")
_ax.set_xlabel("degree k")
_ax.set_ylabel("share with degree ≥ k")
_ax.set_title("the tail is the straight part")
_ax.legend()
_fig.tight_layout()
mo.vstack([
    mo.md(r"""### 🧭 Detour: why fit the tail?

A power law describes the **large-degree tail**. At low degrees the curve
bends away from a straight line — the minimum degree, the early growth, and
finite size all leave their mark there. Including those low degrees drags
the
fitted slope away from the true exponent, so fitting only the tail
```

```
($k \ge 10$) gives the cleaner estimate."""),
    _fig,
])
```

 No description has been provided for this image

In [ ]:
```
# 🧪 Your space to try things out. Write any code here and run it.
# Nothing in this cell is marked, and your tutor does not look at it.
# Start your names with _ (like _small below): such a name belongs to
# this cell only, so it never clashes with the exercise.
_small = ig.Graph(n=3, edges=[(0, 1), (1, 2)])
print(_small.degree())
```

```
[1, 2, 1]
```

Exercise: how robust is a network with hubs?

▶ 📋 Task

In [ ]:
```
# Libraries this cell uses. The notebook has imported them already;
# in your own Python file, install them once and import them:
#   import numpy as np   # pip install numpy==2.5.3

def kappa(g):
    # `k`: the degree of every node in g, as a numpy array.
    k = np.array(g.degree())

    # Return kappa = <k^2> / <k>.
    return np.mean(k**2) / np.mean(k)


def f_c(kappa_value):
    # Critical fraction for random node removal.
    return 1 - 1 / (kappa_value - 1)


# `ba` is your network from the last exercise. `er_net` is a random
# network with the same number of nodes and almost the same number of
# edges.

kappa_ba = kappa(ba)
fc_ba = f_c(kappa_ba)

kappa_er = kappa(er_net)
fc_er = f_c(kappa_er)

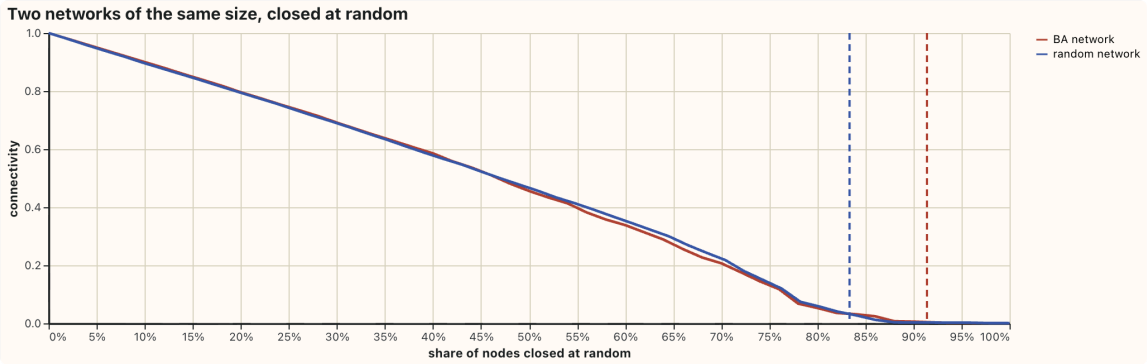
print("BA network:     kappa =", kappa_ba, " f_c =", fc_ba)
print("random network: kappa =", kappa_er, " f_c =", fc_er)

plot_robustness(ba, er_net, fc_ba, fc_er, connectivity, r_index)
```

```
BA network:     kappa = 12.64280947614281  f_c = 0.9141100778081878
random network: kappa = 7.006666666666667  f_c = 0.8335183129855716
```

BA network: f_c = 0.914, R = 0.454

random network: f_c = 0.834, R = 0.449



Each solid line is your connectivity as nodes close at random. The dashed line of the same colour is the f_c your code gives that network.

Submit to Tutor

Submit to Tutor hands this cell in; your tutor answers in the terminal.

Kappa, the critical threshold, and a network with hubs

```
def kappa(g):
    k = np.array(g.degree())
    return np.mean(k ** 2) / np.mean(k)

def f_c(kappa_value):
return 1 - 1 / (kappa_value - 1)
```

$$\kappa = \frac{\langle k^2 \rangle}{\langle k \rangle}, \qquad f_c = 1 - \frac{1}{\kappa - 1}$$

Both come from the degrees alone, without closing a single node. The BA network and the random network have almost the same average degree, but the BA network's hubs make $\langle k^2 \rangle$, and so κ, about twice as large. Its f_c is larger too: under random closure, a network with hubs holds together longer. The formula for f_c is exact only for an infinite random network, so on 2000 nodes it gives the size of the answer, not its decimals. Closing the busiest nodes first is called a targeted attack. It breaks a network with hubs much sooner, and the lecture note covers it.

In [ ]:
```
import numpy as _np

def _connectivity(_g):
    return max(len(c) for c in _g.connected_components()) / _g.vcount()

def _curve_random(_g, n):
    _g2 = _g.copy()
    _order = _np.random.default_rng(0).permutation(_g.vcount())
```

```
    _out = []
    for _i, _v in enumerate(_order):
        _g2.delete_edges(_g2.incident(_v))
        if (_i + 1) % (n // 40) == 0:
            _out.append(((_i + 1) / _g.vcount(), _connectivity(_g2)))
    return _out

def _curve_targeted(_g, n):
    _g2 = _g.copy()
    _out = []
    for _i in range(n):
        _v = _np.argmax(_np.array(_g2.degree()))
        _g2.delete_edges(_g2.incident(_v))
        if (_i + 1) % (n // 40) == 0:
            _out.append(((_i + 1) / _g.vcount(), _connectivity(_g2)))
    return _out

_fig, _ax = plt.subplots(figsize=(6, 3.5))
for _g, _lab, _col in [(ba, "BA", "#3959A6"), (er_net, "random",
"#B14434")]:
    _x, _y = zip(*_curve_targeted(_g, 200))
    _ax.plot(_x, _y, color=_col, label=f"{_lab} targeted", linestyle="--")
    _x, _y = zip(*_curve_random(_g, 200))
    _ax.plot(_x, _y, color=_col, label=f"{_lab} random", linestyle="-")
_ax.set_xlabel("share closed")
_ax.set_ylabel("connectivity")
_ax.set_title("targeted vs random removal")
_ax.legend(fontsize=8)
_fig.tight_layout()
mo.vstack([
    mo.md(r"""### 🧭 Detour: targeted attack flips the story

The hubs hold the BA network together. Remove the **highest-degree** nodes first (targeted) and it collapses almost at once — far sooner than random removal. The random network has no hubs to aim at, so it barely notices the difference."""),
    _fig,
])
```

🧭 Detour: targeted attack flips the story

The hubs hold the BA network together. Remove the **highest-degree** nodes first (targeted) and it collapses almost at once — far sooner than random removal. The random network has no hubs to aim at, so it barely notices the difference.



📋 Session record

Your answer to each question, and every word you typed while working on it — this is what gets reviewed, not the code. Hints are never held against you. **cp0_walkthrough** · pass *Work through the igraph walkthrough and come back when every step shows ✅.*

> Finished. Please proceed.

You typed: "I can see it" · "Finished. Please proceed." *Tutor's note:* They worked through the walkthrough and reported finishing it. **cp1_build** · pass *Build the network, ask it its size, make and edit a copy, and find the new number of the node that was node 2.*

> (no answer — the code is their answer)

Tutor's note: They filled every blank correctly first time: built g, asked it its size with vcount/ecount, copied and edited it, and named was_2_now = 1. **cp2_matrix** · pass *Write the network as an adjacency matrix, sum its rows, ask g for degrees, and find the mean degree.*

> (no answer — the code is their answer)

You typed: "Yes. Since igraph vertex numbers are positions and can change after deleting a vertex, what is the usual way to preserve a node's original identity when we need stable labels across edits? Would we store an attribute such as a name or ID on each vertex?" · "That makes sense. I am ready to move on." *Tutor's note:* They built the adjacency matrix, summed its rows, asked g for its degrees, and averaged them — every blank right first time. **cp3_paths** · pass *List every route from 0 to 6, pick a shortest, count its edges, cut node 4, and measure the pieces.*

> (no answer — the code is their answer)

You typed: "Since adjacency matrices become very inefficient for large sparse networks, is igraph internally using something closer to adjacency lists or another sparse representation when methods like g.degree() are called?" · "Makes sense. I am ready to move on." *Tutor's note:* They listed all routes, picked the [0] shortest route, counted hops as len−1, cut node 4's edges on a copy, and measured the largest component — every blank right first time. **cp4_measure** · pass *Write connectivity(g) and r_index(closed, connectivities).*

> (no answer — the code is their answer)

You typed: "Since every path from nodes on the left to nodes 5 and 6 goes through node 4, would node 4 be considered an articulation point? And is removing all of its incident edges equivalent to deleting the vertex when we are measuring connectivity, except that the isolated node itself remains?" · "Makes sense. Let's move on." *Tutor's note:* They wrote

connectivity as largest group over vcount and r_index as np.mean of connectivities — both right first time, curve on the band. **cp5_grow** · pass *Grow a 2000-node network by preferential attachment.*

> (no answer — the code is their answer)

You typed: "If two networks have the same R-index, could they still have very different robustness behavior? For example, one failing gradually and another staying connected for a long time and then collapsing suddenly?" · "Great. let's move on..." *Tutor's note:* They wrote the full preferential-attachment loop first time: degree array, probabilities, rng.choice without replacement, then add the node and its edges. **cp6_powerlaw** · pass *Build a Barabási–Albert network and read its power-law exponent two ways.*

> (no answer — the code is their answer)

You typed: "Does preferential attachment alone explain the heavy-tailed degree distribution, or is the fact that nodes enter at different times also important because older nodes have had more opportunities to collect edges?" · "thanks. ready to move on." *Tutor's note:* They built ba, computed the cumulative degree share, read alpha as 1 – slope, and fitted powerlaw on degrees 10 and up — all correct first time. **cp7_robust** · pass *Write kappa and f_c, and use them on ba and er_net.*

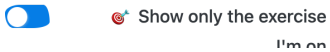
> (no answer — the code is their answer)

You typed: "Why is fitting only the tail generally more reliable for estimating the power-law exponent? Is it because the low-degree part of the distribution does not follow the asymptotic power-law behavior very closely?" · "That makes sense. So the low-degree region reflects finite-size and growth effects, while the tail is the part that more closely follows the asymptotic power-law behavior. Lets ready to move on." *Tutor's note:* They wrote kappa as mean of k squared over mean k and f_c as 1 – 1/(kappa–1), then applied both to ba and er_net — every blank right first time.

🧭 Your own questions (7)

- *Since igraph vertex numbers are positions and can change after deleting a vertex, what is the usual way to preserve a node's original identity when we need stable labels across edits? Would we store an attribute such as a name or ID on each vertex?*
- *Since adjacency matrices become very inefficient for large sparse networks, is igraph internally using something closer to adjacency lists or another sparse representation when methods like g.degree() are called?*
- *Since every path from nodes on the left to nodes 5 and 6 goes through node 4, would node 4 be considered an articulation point? And is removing all of its incident edges equivalent to deleting the vertex when we are measuring connectivity, except that the isolated node itself remains?*

- *If two networks have the same R-index, could they still have very different robustness behavior? For example, one failing gradually and another staying connected for a long time and then collapsing suddenly?*
- *Does preferential attachment alone explain the heavy-tailed degree distribution, or is the fact that nodes enter at different times also important because older nodes have had more opportunities to collect edges?*
- *Why is fitting only the tail generally more reliable for estimating the power-law exponent? Is it because the low-degree part of the distribution does not follow the asymptotic power-law behavior very closely?*
- *Since the BA network is more robust to random deletion because most randomly selected nodes are low-degree nodes, would the opposite happen under targeted deletion, where we always remove the highest-degree node first? Would its robustness then become substantially worse than the random network?*



▶ ℹ️ How the exercises work

All 8 exercises done

